

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Arjun Mallikarjun Banappanavar (1BM24CS052)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Arjun Mallikarjun Banappanavar (**IBM24CS052**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	20
6	Implement 0/1 Knapsack problem using dynamic programming.	24
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	27
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.	30
	(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	33
9	Implement Fractional Knapsack using Greedy technique.	37
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	39
11	Implement "N-Queens Problem" using Backtracking.	42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1:

Write program to obtain the Topological ordering of vertices in a given digraph:

Code:

```
#include <stdio.h>

#define MAX 10

void readGraph(int adj[MAX][MAX], int n) {
    int i, j;

    printf("Enter the adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
}

void findIndegree(int adj[MAX][MAX], int indegree[], int n) {
    int i, j;

    for(i = 0; i < n; i++)
        indegree[i] = 0;

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j])
                indegree[j]++;
        }
    }
}
```

```
}  
}
```

```
void topologicalSort(int adj[MAX][MAX], int n) {
```

```
    int indegree[MAX];
```

```
    int queue[MAX];
```

```
    int front = 0, rear = 0;
```

```
    int i, j, v, count = 0;
```

```
    findIndegree(adj, indegree, n);
```

```
    for(i = 0; i < n; i++) {
```

```
        if(indegree[i] == 0)
```

```
            queue[rear++] = i;
```

```
    }
```

```
    printf("\nTopological sorting result:\n");
```

```
    while(front < rear) {
```

```
        v = queue[front++];
```

```
        printf("%d ", v);
```

```
        count++;
```

```
        for(j = 0; j < n; j++) {
```

```
            if(adj[v][j]) {
```

```
                indegree[j]--;
```

```
            if(indegree[j] == 0)
```

```
                queue[rear++] = j;
```

```

        }
    }
}

if(count != n)
    printf("\nCycle detected in the graph.");

printf("\n");
}

int main() {
    int adj[MAX][MAX];
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    readGraph(adj, n);
    topologicalSort(adj, n);

    return 0;
}

```

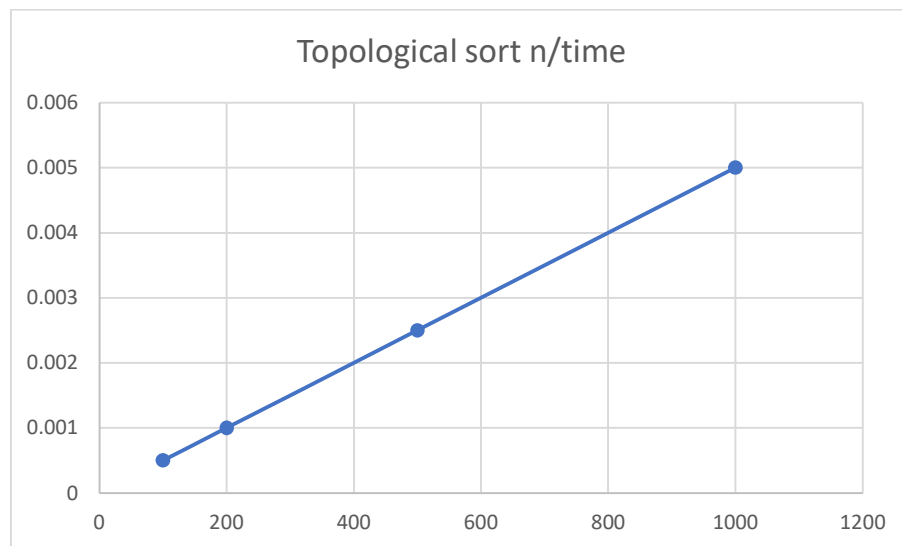
Output:

```
F:\ada_lab\topological_sort.exe
Enter number of vertices: 6
Enter the adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0

Topological sorting result:
0 1 2 3 4 5

Process returned 0 (0x0)   execution time : 58.991 s
Press any key to continue.
```

Time complexity graph:



LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations:

Code:

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

int getMobile(int a[], int dir[], int n) {
    int mobile = 0, mobileIndex = -1;

    for(int i = 0; i < n; i++) {
        if(dir[i] == LEFT && i > 0 &&
            a[i] > a[i - 1] && a[i] > mobile) {
            mobile = a[i];
            mobileIndex = i;
        }

        if(dir[i] == RIGHT && i < n - 1 &&
            a[i] > a[i + 1] && a[i] > mobile) {
            mobile = a[i];
            mobileIndex = i;
        }
    }

    return mobileIndex;
}

void printPermutation(int a[], int n) {
```



```

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

void initialize(int a[], int dir[], int n) {
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
}

void generatePermutations(int a[], int dir[], int n) {
    int mobileIndex, swapIndex, temp;

    printf("\nGenerated permutations:\n");
    printPermutation(a, n);

    while(1) {
        mobileIndex = getMobile(a, dir, n);

        if(mobileIndex == -1)
            break;

        if(dir[mobileIndex] == LEFT)
            swapIndex = mobileIndex - 1;
        else
            swapIndex = mobileIndex + 1;
    }
}

```

```

    temp = a[mobileIndex];
    a[mobileIndex] = a[swapIndex];
    a[swapIndex] = temp;

    temp = dir[mobileIndex];
    dir[mobileIndex] = dir[swapIndex];
    dir[swapIndex] = temp;

    mobileIndex = swapIndex;

    for(int i = 0; i < n; i++) {
        if(a[i] > a[mobileIndex])
            dir[i] = -dir[i];
    }

    printPermutation(a, n);
}
}

int main() {
    int n;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    int a[n], dir[n];

    initialize(a, dir, n);

```

```
generatePermutations(a, dir, n);  
  
return 0;  
}
```

Output:

```
F:\ada_lab\code\johnson_trotter.exe  
Enter the value of n: 3  
  
Generated permutations:  
1 2 3  
1 3 2  
3 1 2  
3 2 1  
2 3 1  
2 1 3  
  
Process returned 0 (0x0)   execution time : 3.275 s  
Press any key to continue.
```

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort:

Code:

```
#include <stdio.h>

#include <time.h>

#define MAX 100000

void merge(int a[], int low, int mid, int high) {
    int temp[MAX];
    int i = low, j = mid + 1, k = 0;

    while(i <= mid && j <= high) {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= mid)
        temp[k++] = a[i++];

    while(j <= high)
        temp[k++] = a[j++];

    for(i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];
}
```

```
void mergeSort(int a[], int low, int high) {  
    if(low < high) {  
        int mid = (low + high) / 2;  
  
        mergeSort(a, low, mid);  
        mergeSort(a, mid + 1, high);  
        merge(a, low, mid, high);  
    }  
}
```

```
void readArray(int a[], int n) {  
    printf("Enter array elements:\n");  
  
    for(int i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
}
```

```
void displayArray(int a[], int n) {  
    printf("\nSorted array:\n");  
  
    for(int i = 0; i < n; i++)  
        printf("%d ", a[i]);  
  
    printf("\n");  
}
```

```
int main() {  
    int n;
```

```
int a[MAX];  
clock_t start, end;  
double time_taken;  
  
printf("Enter size of array: ");  
scanf("%d", &n);  
  
readArray(a, n);  
  
start = clock();  
mergeSort(a, 0, n - 1);  
end = clock();  
  
time_taken = (double)(end - start) / CLOCKS_PER_SEC;  
  
displayArray(a, n);  
printf("Execution time: %f seconds\n", time_taken);  
  
return 0;  
}
```

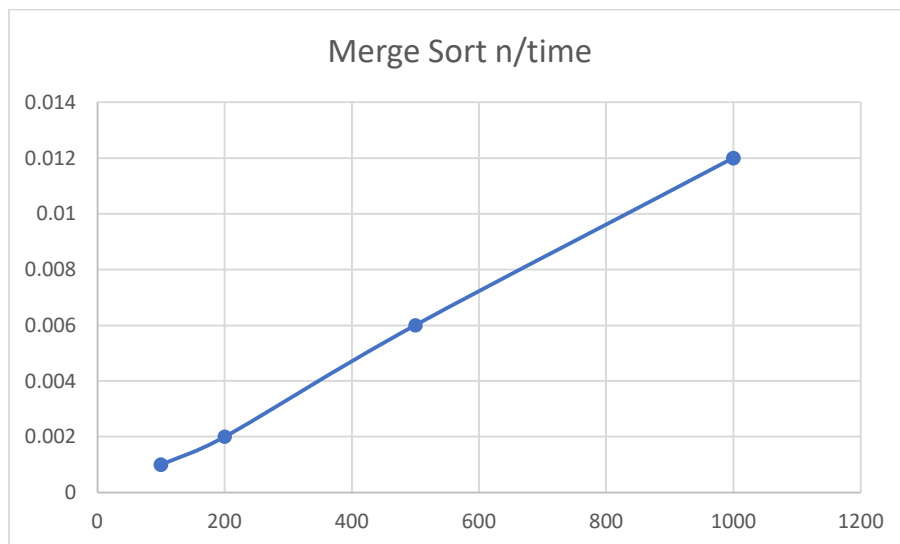
Output:

```
F:\ada_lab\code\merge_sort.exe
Enter size of array: 5
Enter array elements:
5 2 8 1 4

Sorted array:
1 2 4 5 8
Execution time: 0.000000 seconds

Process returned 0 (0x0)   execution time : 14.612 s
Press any key to continue.
```

Time Complexity graph:



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken:

Code:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define MAX 100000
```

```
void quickSort(int a[], int low, int high) {
```

```
    int i, j, pivot, temp;
```

```
    if(low < high) {
```

```
        pivot = low;
```

```
        i = low;
```

```
        j = high;
```

```
        while(i < j) {
```

```
            while(a[i] <= a[pivot] && i < high)
```

```
                i++;
```

```
            while(a[j] > a[pivot])
```

```
                j--;
```

```
            if(i < j) {
```

```
                temp = a[i];
```

```
                a[i] = a[j];
```

```
                a[j] = temp;
```

```
            }
```

```
        }
```



```

        temp = a[pivot];
        a[pivot] = a[j];
        a[j] = temp;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

void readArray(int a[], int n) {
    printf("Enter array elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
}

void displayArray(int a[], int n) {
    printf("\nSorted array:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main() {
    int n;
    int a[MAX];

```

```

    clock_t start, end;
    double time_taken;

    printf("Enter size of array: ");
    scanf("%d", &n);

    readArray(a, n);

    start = clock();
    quickSort(a, 0, n - 1);
    end = clock();

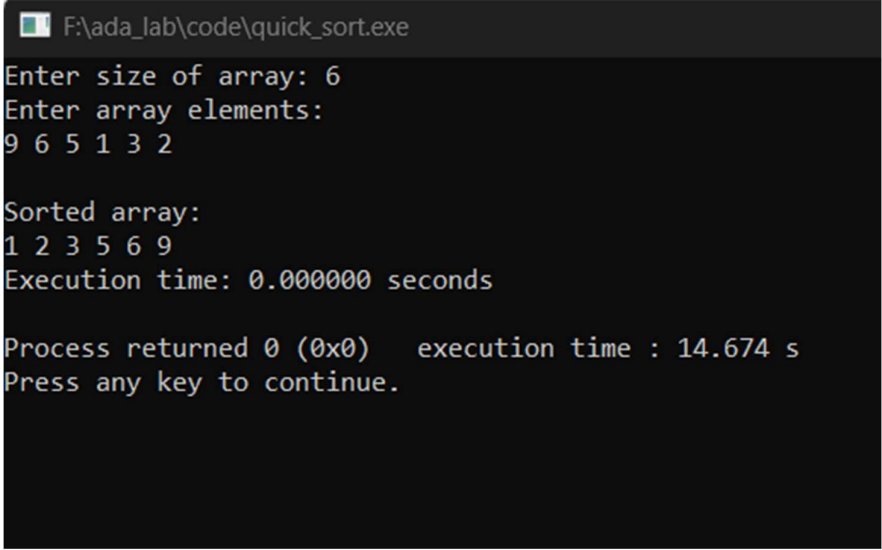
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    displayArray(a, n);
    printf("Execution time: %f seconds\n", time_taken);

    return 0;
}

```

Output:



```

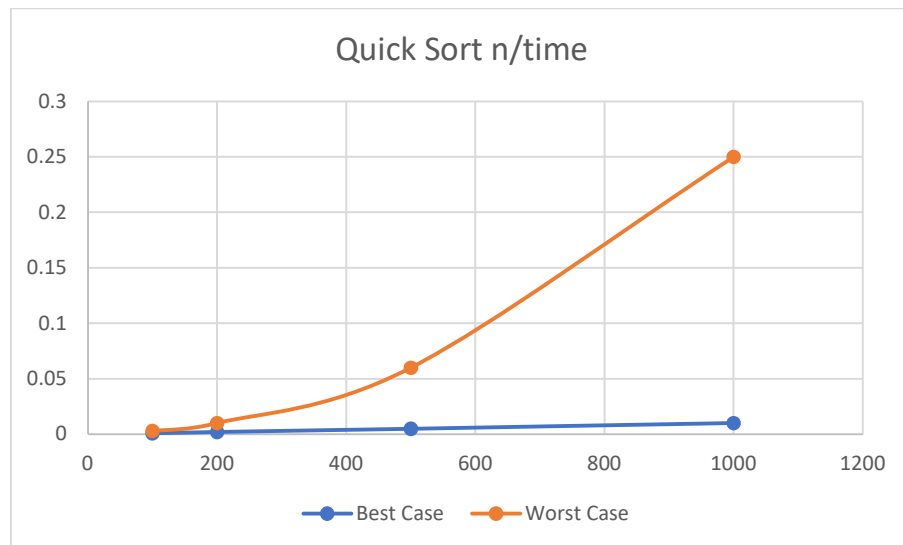
F:\ada_lab\code\quick_sort.exe
Enter size of array: 6
Enter array elements:
9 6 5 1 3 2

Sorted array:
1 2 3 5 6 9
Execution time: 0.000000 seconds

Process returned 0 (0x0)   execution time : 14.674 s
Press any key to continue.

```

Time complexity graph:



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken:

Code:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define MAX 100000
```

```
void heapify(int a[], int n, int i) {
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    int temp;
```

```
    if(left < n && a[left] > a[largest])
```

```
        largest = left;
```

```
    if(right < n && a[right] > a[largest])
```

```
        largest = right;
```

```
    if(largest != i) {
```

```
        temp = a[i];
```

```
        a[i] = a[largest];
```

```
        a[largest] = temp;
```

```
        heapify(a, n, largest);
```

```
    }
```

```
}
```

```

void buildHeap(int a[], int n) {
    for(int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
}

void heapSort(int a[], int n) {
    int temp;

    buildHeap(a, n);

    for(int i = n - 1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

void readArray(int a[], int n) {
    printf("Enter array elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
}

void displayArray(int a[], int n) {
    printf("\nSorted array:\n");

```

```

        for(int i = 0; i < n; i++)
            printf("%d ", a[i]);

        printf("\n");
    }

int main() {
    int n;
    int a[MAX];
    clock_t start, end;
    double time_taken;

    printf("Enter size of array: ");
    scanf("%d", &n);

    readArray(a, n);

    start = clock();
    heapSort(a, n);
    end = clock();

    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    displayArray(a, n);
    printf("Execution time: %f seconds\n", time_taken);

    return 0;
}

```

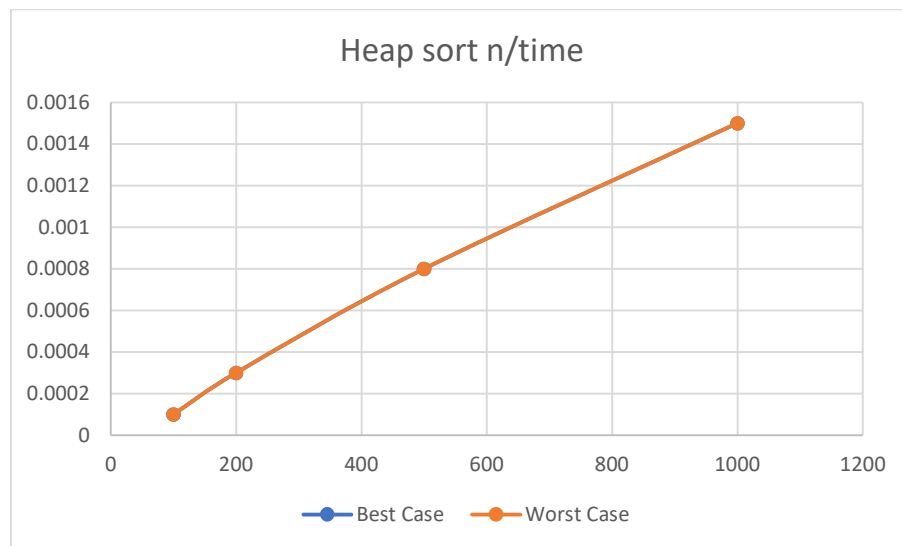
Output:

```
F:\ada_lab\code\heap_sort.exe
Enter size of array: 6
Enter array elements:
12 5 8 1 15 3

Sorted array:
1 3 5 8 12 15
Execution time: 0.000000 seconds

Process returned 0 (0x0)   execution time : 15.608 s
Press any key to continue.
```

Time Complexity graph:



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming:

Code:

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

void readItems(int weight[], int profit[], int n) {
    printf("Enter item weights:\n");
    for(int i = 1; i <= n; i++)
        scanf("%d", &weight[i]);

    printf("Enter item profits:\n");
    for(int i = 1; i <= n; i++)
        scanf("%d", &profit[i]);
}

int knapsack(int n, int weight[], int profit[], int capacity) {
    int dp[n + 1][capacity + 1];

    for(int i = 0; i <= n; i++) {
        for(int j = 0; j <= capacity; j++) {

            if(i == 0 || j == 0)
                dp[i][j] = 0;

            else if(weight[i] <= j)
```



```

        dp[i][j] = max(
            profit[i] + dp[i - 1][j - weight[i]],
            dp[i - 1][j]
        );

    else
        dp[i][j] = dp[i - 1][j];
    }
}

return dp[n][capacity];
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int weight[n + 1], profit[n + 1];

    readItems(weight, profit, n);

    printf("Enter bag capacity: ");
    scanf("%d", &capacity);

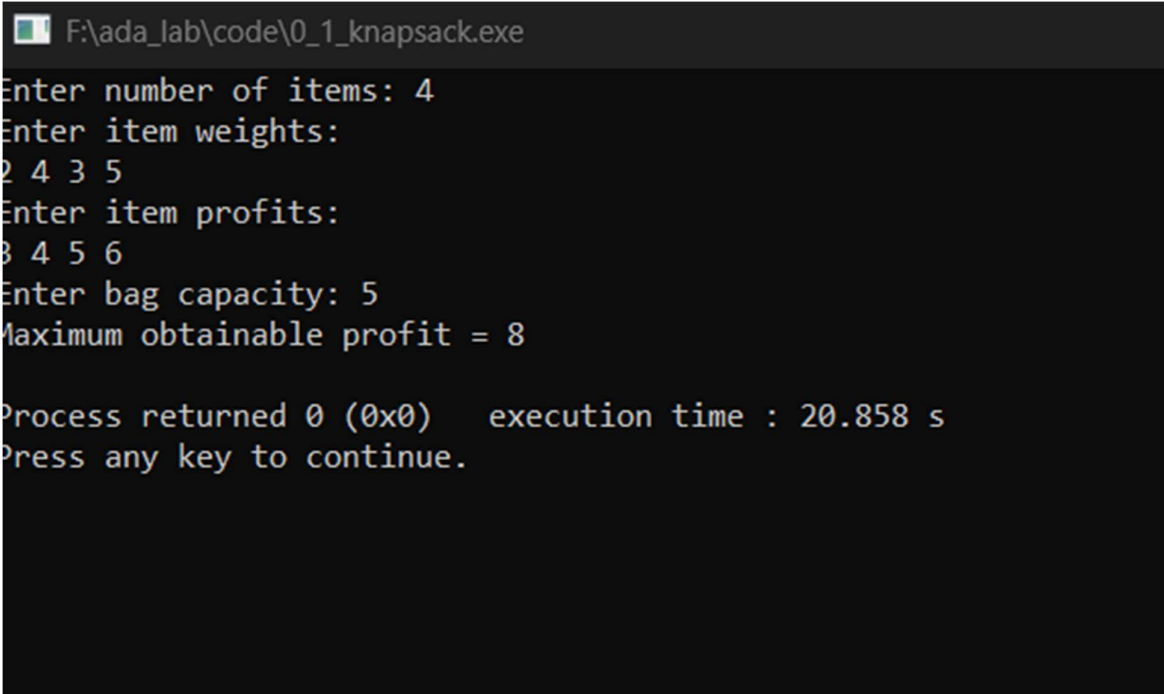
    int result = knapsack(n, weight, profit, capacity);

    printf("Maximum obtainable profit = %d\n", result);
}

```

```
    return 0;  
}
```

Output:



```
F:\ada_lab\code\0_1_knapsack.exe  
Enter number of items: 4  
Enter item weights:  
2 4 3 5  
Enter item profits:  
8 4 5 6  
Enter bag capacity: 5  
Maximum obtainable profit = 8  
  
Process returned 0 (0x0)   execution time : 20.858 s  
Press any key to continue.
```

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm:

Code:

```
#include <stdio.h>

#define MAX 10

#define INF 999

void readMatrix(int cost[MAX][MAX], int n) {

    printf("Enter the cost matrix:\n");

    for(int i = 0; i < n; i++) {

        for(int j = 0; j < n; j++) {

            scanf("%d", &cost[i][j]);

        }

    }

}

void displayMatrix(int cost[MAX][MAX], int n) {

    printf("\nShortest path matrix:\n");

    for(int i = 0; i < n; i++) {

        for(int j = 0; j < n; j++) {

            if(cost[i][j] == INF)

                printf("INF ");

            else

                printf("%d ", cost[i][j]);

        }

    }

}
```

```

    }

    printf("\n");
}

}

void floydWarshall(int cost[MAX][MAX], int n) {

    for(int k = 0; k < n; k++) {

        for(int i = 0; i < n; i++) {

            for(int j = 0; j < n; j++) {

                if(cost[i][k] + cost[k][j] < cost[i][j])

                    cost[i][j] = cost[i][k] + cost[k][j];

            }

        }

    }

    displayMatrix(cost, n);

}

int main() {

    int n;

    int cost[MAX][MAX];

    printf("Enter number of vertices: ");

    scanf("%d", &n);

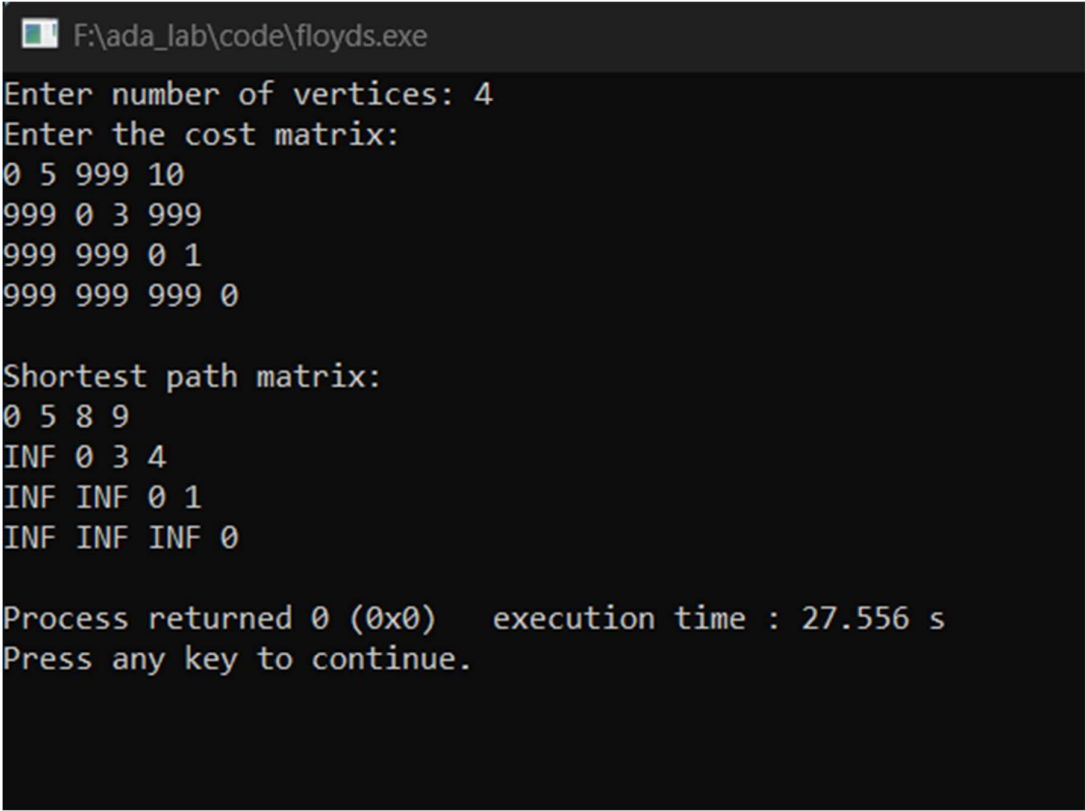
    readMatrix(cost, n);

    floydWarshall(cost, n);

```

```
    return 0;  
}
```

Output:



```
F:\ada_lab\code\floyds.exe  
Enter number of vertices: 4  
Enter the cost matrix:  
0 5 999 10  
999 0 3 999  
999 999 0 1  
999 999 999 0  
  
Shortest path matrix:  
0 5 8 9  
INF 0 3 4  
INF INF 0 1  
INF INF INF 0  
  
Process returned 0 (0x0)   execution time : 27.556 s  
Press any key to continue.
```

LAB PROGRAM 8A:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm:

Code:

```
#include <stdio.h>

#define MAX 10
#define INF 999

void readMatrix(int cost[MAX][MAX], int n) {
    printf("Enter the adjacency matrix:\n");

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
}

void primMST(int cost[MAX][MAX], int n) {
    int visited[MAX] = {0};
    int min, u = 0, v = 0;
    int mincost = 0;
    int edgeCount = 1;

    visited[0] = 1;

    printf("\nEdges selected for MST:\n");

    while(edgeCount < n) {
```

```

    min = INF;

    for(int i = 0; i < n; i++) {
        if(visited[i]) {
            for(int j = 0; j < n; j++) {
                if(!visited[j] && cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }

    printf("%d -> %d : %d\n", u, v, min);

    mincost += min;
    visited[v] = 1;
    edgeCount++;
}

printf("Total cost of MST = %d\n", mincost);
}

int main() {
    int n;
    int cost[MAX][MAX];

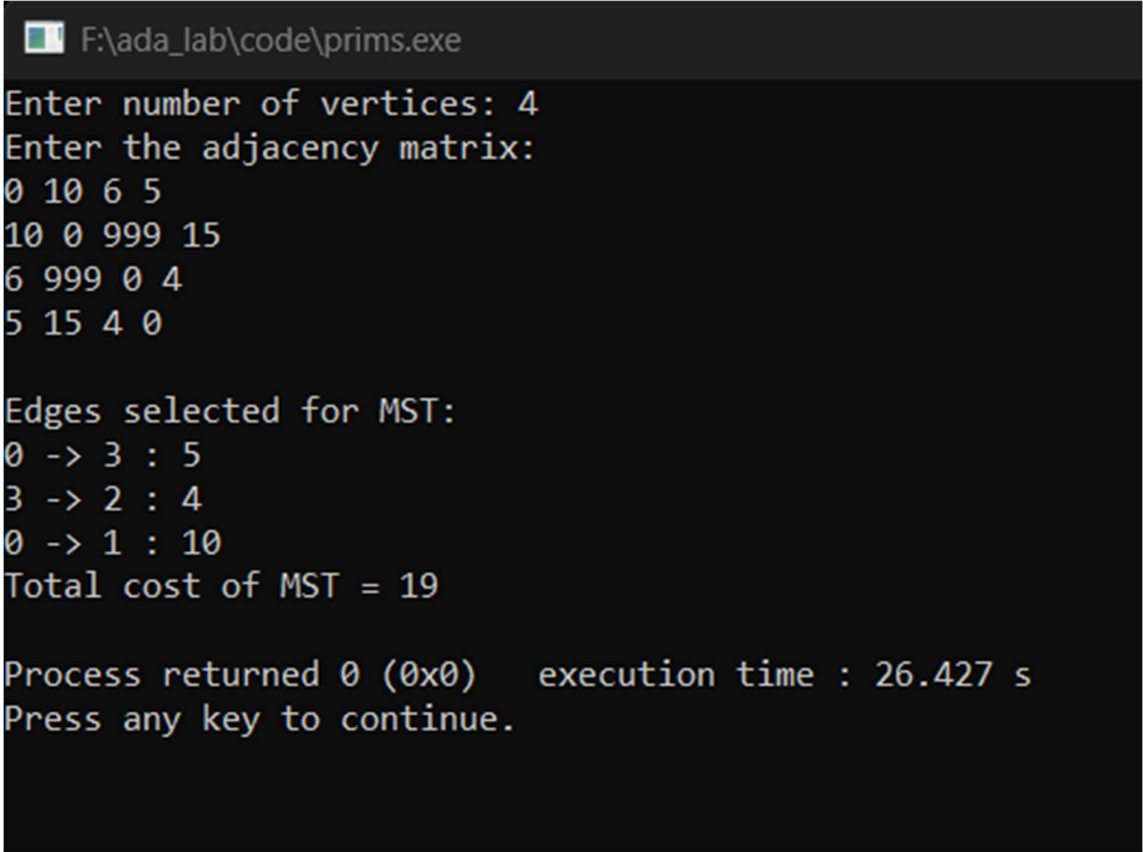
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    readMatrix(cost, n);

```

```
    primMST(cost, n);  
  
    return 0;  
}
```

Output:



```
F:\ada_lab\code\prims.exe  
Enter number of vertices: 4  
Enter the adjacency matrix:  
0 10 6 5  
10 0 999 15  
6 999 0 4  
5 15 4 0  
  
Edges selected for MST:  
0 -> 3 : 5  
3 -> 2 : 4  
0 -> 1 : 10  
Total cost of MST = 19  
  
Process returned 0 (0x0)   execution time : 26.427 s  
Press any key to continue.
```


LAB PROGRAM 8B:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm:

Code:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 999
```

```
int parent[MAX];
```

```
int find(int i) {  
    while(parent[i])  
        i = parent[i];
```

```
    return i;  
}
```

```
int unionSets(int i, int j) {  
    if(i != j) {  
        parent[j] = i;  
        return 1;  
    }
```

```
    return 0;  
}
```

```
void readMatrix(int cost[MAX][MAX], int n) {  
    printf("Enter the adjacency matrix:\n");  
  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < n; j++) {
```

```

        scanf("%d", &cost[i][j]);

        if(cost[i][j] == 0)
            cost[i][j] = INF;
    }
}

void kruskalMST(int cost[MAX][MAX], int n) {
    int a, b, u, v;
    int min, mincost = 0;
    int edgeCount = 1;

    printf("\nEdges selected for MST:\n");

    while(edgeCount < n) {
        min = INF;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(cost[i][j] < min) {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }

        u = find(u);
        v = find(v);

        if(unionSets(u, v)) {

```

```

        printf("%d -> %d : %d\n", a, b, min);
        mincost += min;
        edgeCount++;
    }

    cost[a][b] = cost[b][a] = INF;
}

printf("Total cost of MST = %d\n", mincost);
}

int main() {
    int n;
    int cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    readMatrix(cost, n);
    kruskalMST(cost, n);

    return 0;
}

```

Output:

```
F:\ada_lab\code\kruskals.exe
Enter number of vertices: 4
Enter the adjacency matrix:
0 10 6 5
10 0 999 15
6 999 0 4
5 15 4 0

Edges selected for MST:
2 -> 3 : 4
0 -> 3 : 5
0 -> 2 : 6
Total cost of MST = 15

Process returned 0 (0x0)   execution time : 24.875 s
Press any key to continue.
```

LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique:

Code:

```
#include <stdio.h>

struct Item {
    int weight;
    int profit;
    float ratio;
};

void readItems(struct Item item[], int n) {
    printf("Enter weight and profit for each item:\n");

    for(int i = 0; i < n; i++) {
        scanf("%d%d", &item[i].weight, &item[i].profit);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }
}

void sortItems(struct Item item[], int n) {
    struct Item temp;

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(item[j].ratio < item[j + 1].ratio) {
                temp = item[j];
                item[j] = item[j + 1];
                item[j + 1] = temp;
            }
        }
    }
}

void fractionalKnapsack(struct Item item[], int n, int capacity) {
    float totalProfit = 0.0;

    sortItems(item, n);

    for(int i = 0; i < n; i++) {
        if(capacity >= item[i].weight) {
            capacity -= item[i].weight;
            totalProfit += item[i].profit;
        }
        else {
            totalProfit += item[i].ratio * capacity;
            break;
        }
    }
}
```

```

    }
}

printf("Maximum obtainable profit = %.2f\n", totalProfit);
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item item[n];

    readItems(item, n);

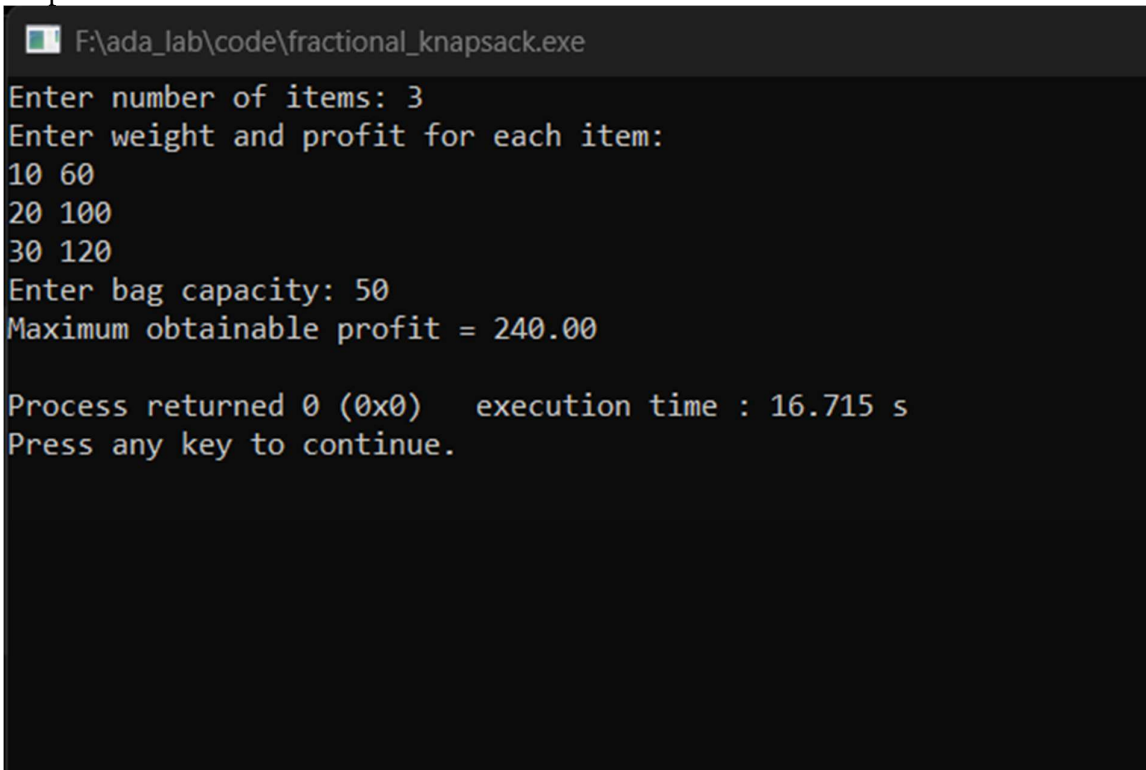
    printf("Enter bag capacity: ");
    scanf("%d", &capacity);

    fractionalKnapsack(item, n, capacity);

    return 0;
}

```

Output:



```

F:\ada_lab\code\fractional_knapsack.exe
Enter number of items: 3
Enter weight and profit for each item:
10 60
20 100
30 120
Enter bag capacity: 50
Maximum obtainable profit = 240.00

Process returned 0 (0x0)   execution time : 16.715 s
Press any key to continue.

```

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm:

Code:

```
#include <stdio.h>

#define MAX 10
#define INF 999

void readMatrix(int cost[MAX][MAX], int n) {
    printf("Enter the adjacency matrix:\n");

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
}

void displayDistances(int distance[], int n, int source) {
    printf("\nShortest distances from source %d:\n", source);

    for(int i = 0; i < n; i++)
        printf("%d -> %d : %d\n", source, i, distance[i]);
}

void dijkstra(int cost[MAX][MAX], int n, int source) {
    int distance[MAX];
    int visited[MAX] = {0};
    int min, next;

    for(int i = 0; i < n; i++)
        distance[i] = cost[source][i];

    distance[source] = 0;
    visited[source] = 1;

    for(int i = 1; i < n; i++) {
        min = INF;

        for(int j = 0; j < n; j++) {
            if(!visited[j] && distance[j] < min) {
                min = distance[j];
                next = j;
            }
        }
    }
}
```

```

    }

    visited[next] = 1;

    for(int j = 0; j < n; j++) {
        if(!visited[j] &&
            min + cost[next][j] < distance[j]) {
            distance[j] = min + cost[next][j];
        }
    }
}

displayDistances(distance, n, source);
}

int main() {
    int n, source;
    int cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    readMatrix(cost, n);

    printf("Enter source vertex: ");
    scanf("%d", &source);

    dijkstra(cost, n, source);

    return 0;
}

```


Output:

```
F:\ada_lab\code\dijkstra.exe
Enter number of vertices: 4
Enter the adjacency matrix:
0 1 4 0
1 0 2 6
4 2 0 3
0 6 3 0
Enter source vertex: 0

Shortest distances from source 0:
0 -> 0 : 0
0 -> 1 : 1
0 -> 2 : 3
0 -> 3 : 6

Process returned 0 (0x0)   execution time : 26.333 s
Press any key to continue.
```

LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking:

Code:

```
#include <stdio.h>
#include <stdlib.h>

int x[20];
int count = 0;

int place(int row, int col) {
    for(int i = 1; i < row; i++) {
        if(x[i] == col || abs(x[i] - col) == abs(i - row))
            return 0;
    }

    return 1;
}

void printBoard(int n) {
    printf("\nArrangement %d:\n", count);

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            if(x[i] == j)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}

void nQueens(int row, int n) {
    for(int col = 1; col <= n; col++) {
        if(place(row, col)) {
            x[row] = col;

            if(row == n) {
                count++;
                printBoard(n);
            }
            else {
                nQueens(row + 1, n);
            }
        }
    }
}
```

```

int main() {
    int n;

    printf("Enter value of N: ");
    scanf("%d", &n);

    nQueens(1, n);

    if(count == 0)
        printf("No valid solution found.");

    return 0;
}

```

Output:

```

F:\ada_lab\code\n_queens.exe
Enter value of N: 4

Arrangement 1:
. Q . .
. . . Q
Q . . .
. . Q .

Arrangement 2:
. . Q .
Q . . .
. . . Q
. Q . .

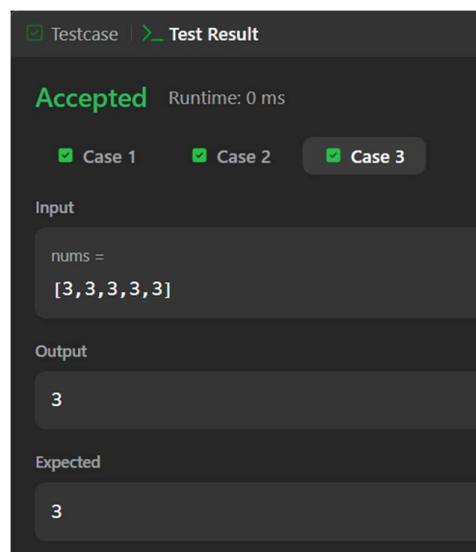
Process returned 0 (0x0)   execution time : 5.563 s
Press any key to continue.

```

LEET CODE PROBLEMS:

287. Find the Duplicate Number:

```
int findDuplicate(int* nums, int numsSize) {  
    int slow = nums[0];  
    int fast = nums[0];  
  
    do {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
    } while (slow != fast);  
  
    slow = nums[0];  
  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[fast];  
    }  
  
    return slow;  
}
```



Testcase **Test Result**

Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2
 ☒ Case 3

Input

nums =
[3,1,3,4,2]

Output

3

Expected

3

Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2
 ☒ Case 3

Input

nums =
[1,3,4,2,2]

Output

2

Expected

2

Description Editorial Solutions Submissions **Accepted** X

← All Submissions

Accepted 59 / 59 testcases passed

● Anjun_872 submitted at Jun 06, 2026 18:23

Analysis Solution

Runtime

4 ms | Beats: 62.76%

Memory

14.89 MB | Beats: 81.27%

Code | C

```

1 int findDuplicate(int* nums, int numsSize) {
2     int slow = nums[0];
3     int fast = nums[0];
  
```

207. Course Schedule:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int indegree[numCourses];
    memset(indegree, 0, sizeof(indegree));

    int adj[numCourses][numCourses];
    memset(adj, 0, sizeof(adj));

    for(int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];

        adj[b][a] = 1;
        indegree[a]++;
    }

    int queue[numCourses];
    int front = 0, rear = 0;

    for(int i = 0; i < numCourses; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }

    int count = 0;
```

```

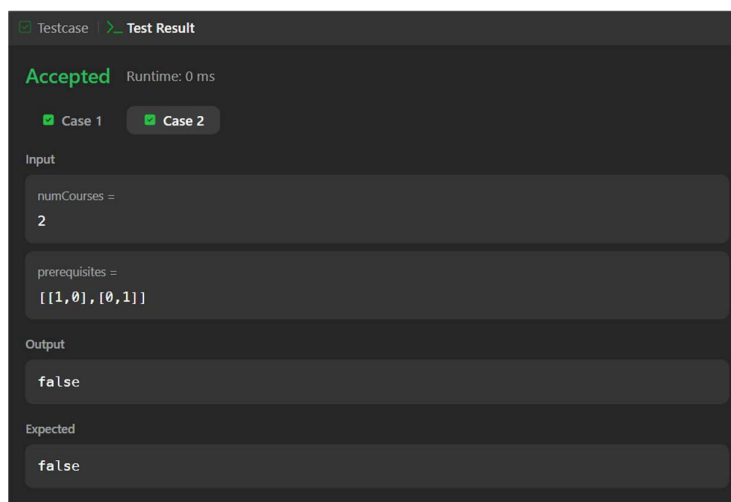
while(front < rear) {
    int node = queue[front++];
    count++;

    for(int i = 0; i < numCourses; i++) {
        if(adj[node][i]) {
            indegree[i]--;

            if(indegree[i] == 0)
                queue[rear++] = i;
        }
    }
}

return count == numCourses;
}

```



Testcase > Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Description Accepted x Editorial Solutions Submissions

← All Submissions

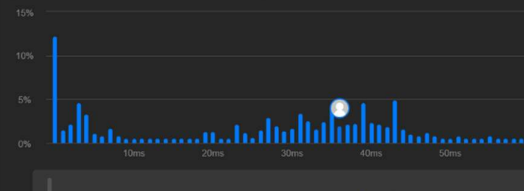
Accepted 54 / 54 testcases passed

Arjun_872 submitted at Jun 05, 2026 23:26

Analysis Solution

Runtime 36 ms | Beats: 38.01%

Memory 28.78 MB | Beats: 57.36%



Code C

```
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, i
2 int indegree[numCourses]);
3 memset(indegree, 0, sizeof(indegree));
```


801. Minimum Swaps To Make Sequences Increasing:

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {  
    int keep = 0;  
    int swp = 1;  
  
    for(int i = 1; i < nums1Size; i++) {  
        int nKeep = 1000000;  
        int nSwap = 1000000;  
  
        if(nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1]) {  
            nKeep = keep;  
            nSwap = swp + 1;  
        }  
  
        if(nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {  
            if(swp < nKeep)  
                nKeep = swp;  
  
            if(keep + 1 < nSwap)  
                nSwap = keep + 1;  
        }  
  
        keep = nKeep;  
        swp = nSwap;  
    }  
  
    return keep < swp ? keep : swp;  
}
```

}

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums1 =
[0, 3, 5, 8, 9]

nums2 =
[2, 1, 4, 6, 9]

Output

1

Expected

1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums1 =
[1, 3, 5, 4]

nums2 =
[1, 2, 3, 7]

Output

1

Expected

1

Code | Accepted

All Submissions

Accepted 117 / 117 testcases passed

Arjun_872 submitted at Jun 06, 2026 00:03

Analysis Solution

Runtime

6 ms Beats 13.64%

Memory

16.23 MB Beats 98.48%



Code | C

```
1 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {  
2     int keep = 0;  
3     int swp = 1;
```